

编译原理作业 5

姓名：梁力航

学号：23336128

日期：2026 年 6 月 17 日

第 1 题

1.1 公共子表达式消除

对基本块消除公共子表达式。左边为原指令，右边注明优化后调整的指令（未变指令不重写）。

原指令	优化后
(1) $g = a + m$	
(2) $q = b * c$	
(3) $u = a + m$	$u = g$ ($a + m$ 已由 (1) 计算)
(4) $d = a * a$	
(5) $a = 4 + b$	
(6) $h = b * c$	$h = q$ ($b * c$ 已由 (2) 计算)

说明：子表达式 $a + m$ 在 (1) 和 (3) 中重复出现， $b * c$ 在 (2) 和 (6) 中重复出现。消除公共子表达式后，(3) 直接复用 g ，(6) 直接复用 q 。注意 (5) 修改了 a 的值，但 $g = a + m$ 在 (5) 之前已经计算，所以 (3) 可以安全地使用 g 。

1.2 复制传播、常量折叠与代数化简

反复施用复制传播、常量折叠和代数化简。优化后的指令在右边注明。

原指令	优化后
(1) $c = 4$	
(2) $t = a$	
(3) $m = b + t$	$m = b + a$ (复制传播: $t \rightarrow a$)
(4) $p = c * c$	$p = 16$ (常量折叠: $4 \times 4 = 16$)
(5) $i = x * 0$	$i = 0$ (代数化简: $x \times 0 = 0$)
(6) $n = m * 2$	
(7) $e = c + p$	$e = 20$ (常量折叠: $4 + 16 = 20$)
(8) $r = e + n$	

优化过程说明：

- 第 (2)(3) 步——复制传播： $t = a$ ，因此将 $m = b + t$ 中的 t 替换为 a ，得到 $m = b + a$ 。
- 第 (4) 步——常量折叠：已知 $c = 4$ （常量），故 $p = c * c = 4 \times 4 = 16$ 。
- 第 (5) 步——代数化简：利用代数恒等式 $x \times 0 = 0$ ，将 $i = x * 0$ 简化为 $i = 0$ 。
- 第 (7) 步——常量折叠：已知 $c = 4$ 、 $p = 16$ ，故 $e = c + p = 4 + 16 = 20$ 。
- 第 (8) 步在优化后 $r = e + n = 20 + n$ ，其中 $n = m * 2$ 无法进一步简化（ m 非编译时常量）。

1.3 活跃变量分析

基本块入口活跃变量集为 $\{x\}$ ，出口为 $\{s, e\}$ ，且基本块不包含死代码。采用向后流分析（backward flow analysis），从出口向前逐条计算每个程序点的活跃变量集。

活跃变量计算规则：对于指令 $d = u \text{ op } v$ （或 $d = u$ ），

$$\text{IN} = (\text{OUT} \setminus \{d\}) \cup \{u, v\}$$

（若操作数为常量则可忽略，因其不在变量集合中。）

活跃变量分析	活跃变量集
	$\{x\}$
(1) $b = x * 0$	
	$\{b\}$
(2) $n = b$	
	$\{b, n\}$
(3) $a = b * 2$	
	$\{a, n\}$
(4) $e = a + n$	
	$\{e\}$
(5) $s = 2 + 3$	
	$\{s, e\}$

计算步骤（从下往上）：

1. 出口处 $\{s, e\}$ 。第 (5) 条 $s = 2 + 3$ 定义 s ，无变量使用，故 (5) 前为 $\{e\}$ 。
2. 第 (4) 条 $e = a + n$ 定义 e ，使用 a, n ，故 (4) 前为 $\{a, n\}$ 。
3. 第 (3) 条 $a = b * 2$ 定义 a ，使用 b ，故 (3) 前为 $\{b, n\}$ 。
4. 第 (2) 条 $n = b$ 定义 n ，使用 b ，故 (2) 前为 $\{b\}$ 。
5. 第 (1) 条 $b = x * 0$ 定义 b ，使用 x ，故 (1) 前为 $\{x\}$ ，与入口一致。✓

第 2 题

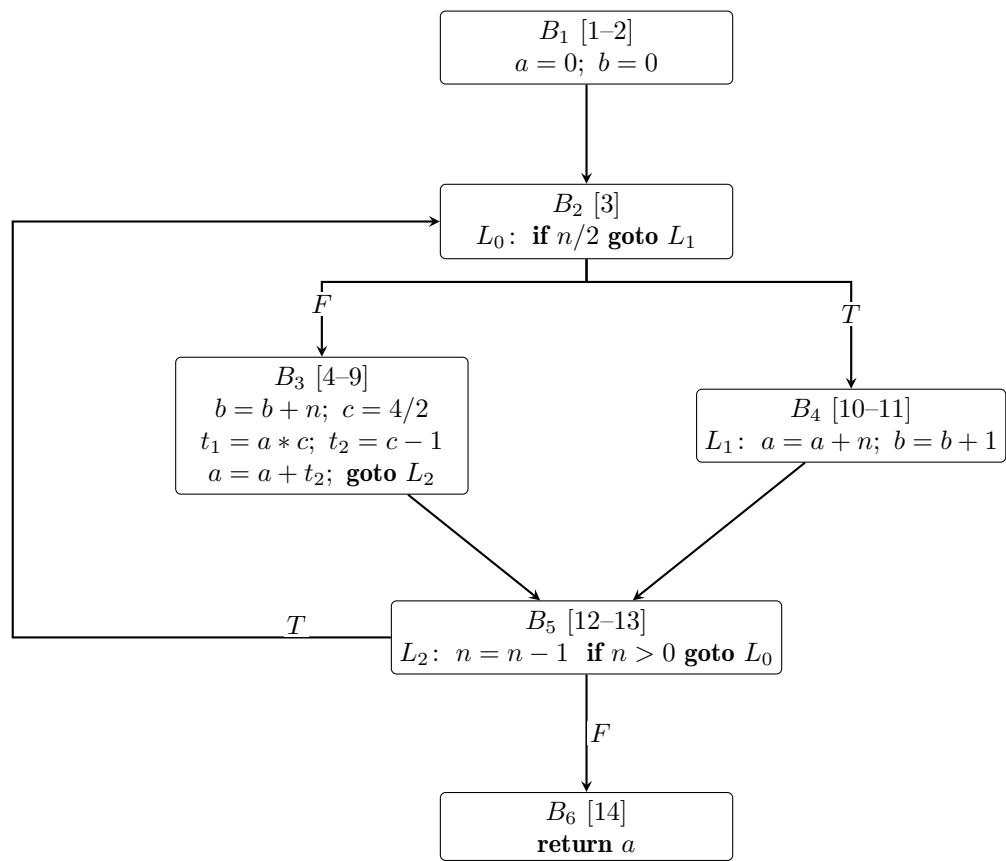
2.1 基本块划分与流图

基本块划分

基本块的首指令（leader）规则：(1) 第一条指令是首指令；(2) 跳转目标指令是首指令；(3) 紧跟在跳转指令之后的指令是首指令。

基本块	指令范围	内容简述
B_1	1-2	$a = 0; b = 0$
B_2	3	L_0 : if $n/2$ goto L_1
B_3	4-9	$b = b + n; c = 4/2; t_1 = a * c; t_2 = c - 1; a = a + t_2$; goto L_2
B_4	10-11	L_1 : $a = a + n; b = b + 1$
B_5	12-13	L_2 : $n = n - 1$; if $n > 0$ goto L_0
B_6	14	return a

流图 (Flow Graph)



说明：

- B_2 中的指令 3 以 $n/2$ 为条件：当 $n/2 \neq 0$ （即 $n \geq 2$ 或 $n \leq -2$ ）时跳转到 L_1 (B_4)；否则顺序进入 B_3 。
- B_3 以无条件跳转 **goto** L_2 结束，进入 B_5 。

- B_4 执行完后自然落入下方的 L_2 (B_5)。
- B_5 以 $n > 0$ 为条件：若真，回跳到 L_0 (B_2)；若假，进入 B_6 返回。

2.2 DAG 有向无环图

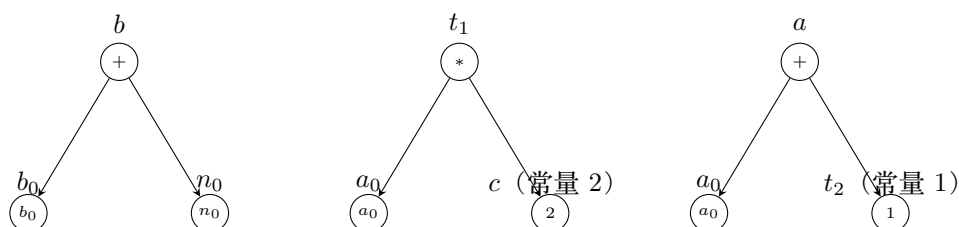
将代码片段 4–8 转换为 DAG：

```

4:   $b = b + n$ 
5:   $c = 4 / 2$ 
6:   $t_1 = a * c$ 
7:   $t_2 = c - 1$ 
8:   $a = a + t_2$ 

```

DAG 构造过程中，常量表达式 $4/2 = 2$ 和 $2 - 1 = 1$ 可被常量折叠，因此 c 直接指向常量节点 2， t_2 直接指向常量节点 1。



DAG 节点与指令的对应关系：

指令	DAG 节点	说明
4: $b = b + n$	+ 节点 (子节点 b_0, n_0), 结果标为 b	加法运算，无常量折叠
5: $c = 4 / 2$	常量 2, 标为 c	常量折叠: $4/2 = 2$
6: $t_1 = a * c$	* 节点 (子节点 $a_0, 2$), 结果标为 t_1	c 已折叠为常量 2
7: $t_2 = c - 1$	常量 1, 标为 t_2	常量折叠: $2 - 1 = 1$
8: $a = a + t_2$	+ 节点 (子节点 $a_0, 1$), 结果标为 a	t_2 已折叠为常量 1

说明：

- 叶节点 b_0 、 n_0 、 a_0 分别表示变量 b 、 n 、 a 的初始值。
- 指令 5 的 $c = 4/2$ 在 DAG 构造时被常量折叠， c 直接关联常量节点 2。
- 指令 7 的 $t_2 = c - 1 = 2 - 1$ 同样被常量折叠， t_2 直接关联常量节点 1。
- 指令 6 的 $t_1 = a_0 * c = a_0 * 2$ ，使用常量节点 2。
- 指令 8 的 $a = a_0 + t_2 = a_0 + 1$ ，使用常量节点 1。
- a_0 在 DAG 中出现两次（被 t_1 和新的 a 引用），体现了 DAG 的共享特性。
- 注意：最终 a 的新值取决于 t_2 而 $t_2 = 1$ ，因此 $a = a_0 + 1$ ，而非 $a = a_0 + t_2$ 。

2.3 代码优化方法

对代码片段 4-8，可以应用以下两种优化方法：

方法一：常量折叠 (Constant Folding)

将编译时可计算出结果的常量表达式直接替换为其值。

原指令	优化后	说明
$c = 4 / 2$	$c = 2$	$4/2$ 是编译时常量，直接计算为 2
$t_2 = c - 1$	$t_2 = 1$	c 已折叠为 2, $2 - 1 = 1$ 也是编译时常量

方法二：常量传播 (Constant Propagation)

将已知的常量值传播到使用该变量的指令中，以发现更多优化机会。

原指令	优化后	说明
$t_1 = a * c$	$t_1 = a * 2$	c 已折叠为常量 2，传播到此处
$a = a + t_2$	$a = a + 1$	t_2 已折叠为常量 1，传播到此处

优化效果：经过常量折叠和常量传播后，原 5 条指令中的 4 条得到了简化，消除了两次除法、一次减法和两次不必要变量引用，指令执行效率明显提高。